Lab 1:
UNIX & Git Basics

EC327

Introduction to Software Engineering

Spring 2020

OVERVIEW

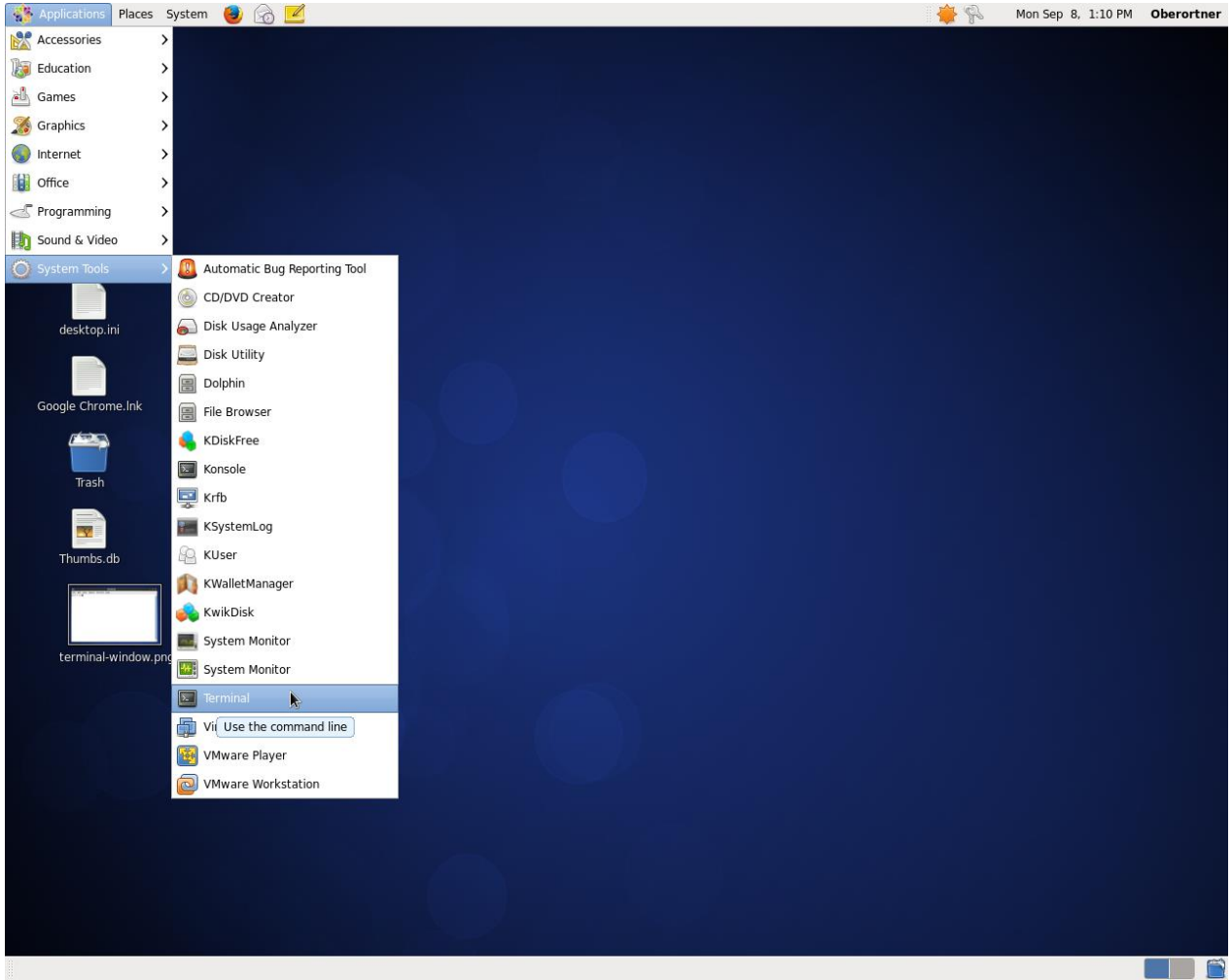
- **Introduction** to UNIX
- The **UNIX Shell**
- **Listing** Files and Directories
- **Navigating** through the File/Directory Tree
- Create, Copy, Move, Delete Files
- File **Editors**
- **Security and Access Rights**
- **“Man pages”**
- **Zipping** Files and Directories
- **Motivation/Purpose**
- “Little” **Assignment**

Introduction to UNIX

- Unix is a Computer Operating System (OS)
- Developed at AT&T Bell Labs
  - Started in the late 60s
  - Ken Thompson and Dennis Ritchie
- Implemented in **C** and Assembly
- Prominent **Derivatives** of UNIX:
  - GNU/Linux
  - MacOS X
  - CentOS
- **Textual** UI: Shell
- **Graphical** UI: X Windows System (X11)

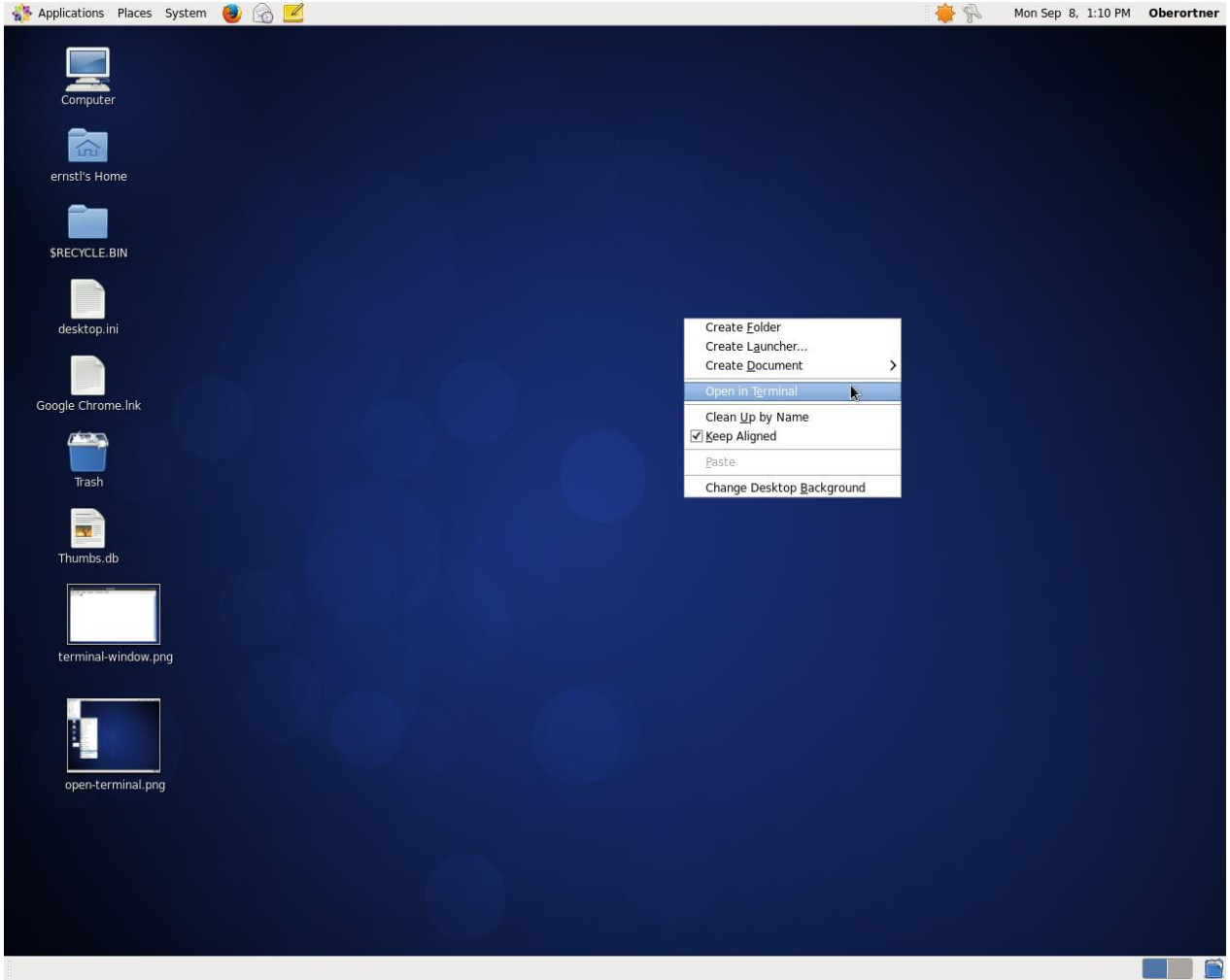
The UNIX Shell

- Textual UI to the OS Kernel
- Accessible via **Terminal** tool



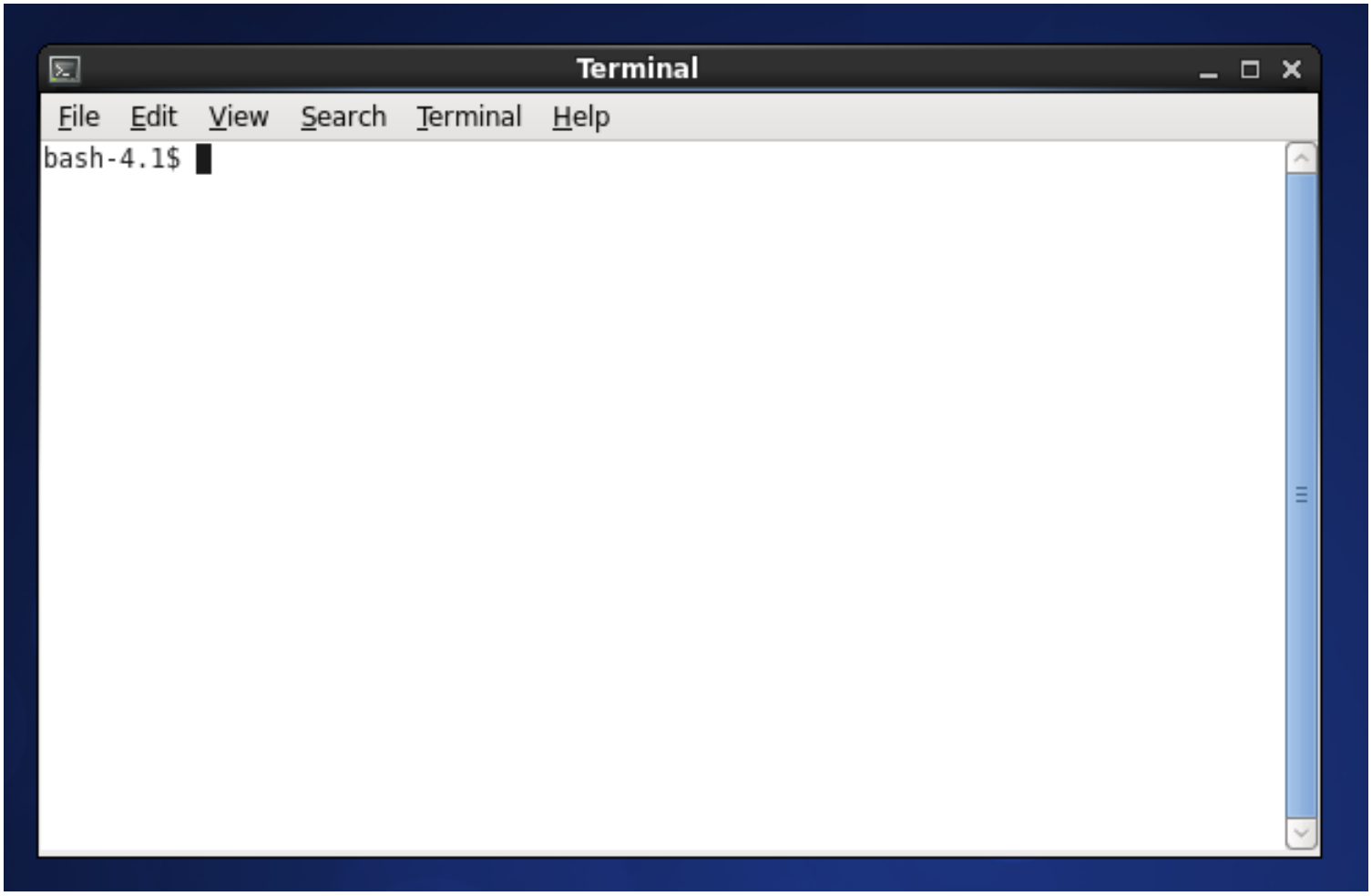
The UNIX Shell

- Textual UI to the OS Kernel
- Accessible via **Terminal** tool



The UNIX Shell

- Textual UI to the OS Kernel
- Accessible via **Terminal** tool



The UNIX Shell

- Textual UI to the OS Kernel
- Accessible via **Terminal** tool
- **Nice Features** of Terminal/Shell:
  - **History**: Up/down arrows
  - **Auto-completion**: Tab

Mostly every command takes **arguments**

```
<command> [<list-of-arguments>]*
```

Examples:

```
ls -l
cd ./my/directory
cp file1 file2
```

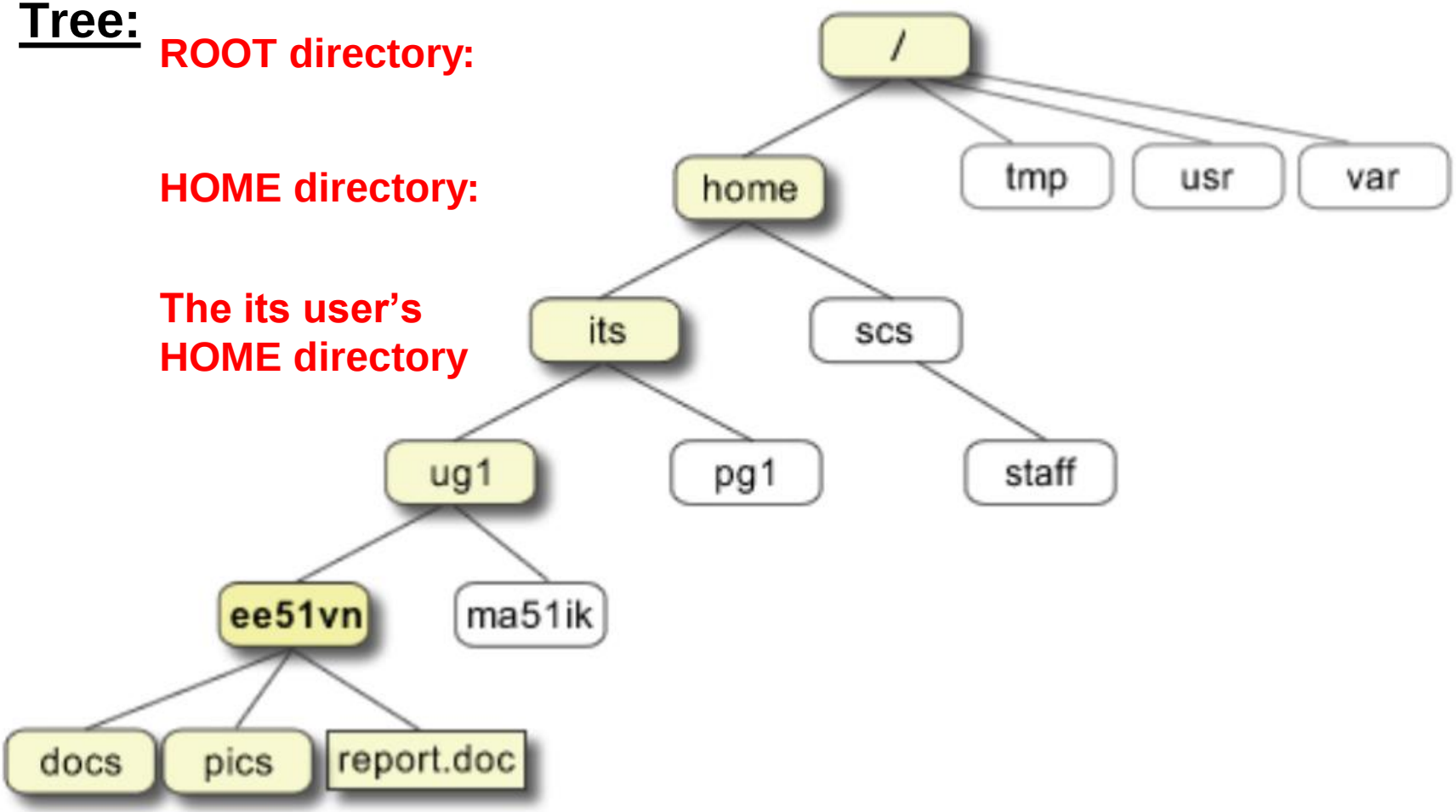
Files and Directories

The UNIX File Tree:

ROOT directory:

HOME directory:

The its user's HOME directory



Listing Files

- `ls` command **list** files and directories

- `ls -l`
  - list files, one per line
- `ls -a`
  - list all, including hidden files and directories
  - Names of hidden files start with **.**
- `ls -lha`
  - list all files, one per line, human readable sizes

Navigating through Directories

- `cd` command **c**hange **d**irectory

`cd .` Go to the current directory

`cd ..` Go to the directory one level above

`cd` Go to the HOME directory

`cd -` Go to the previous location

`cd /absolute/path/to/directory`

ROOT Directory

`cd ./relative/path/to/directory`

Current Directory

- `pwd` command **p**rint **w**orking **d**irectory

Create, Copy, Move, Delete Files and Directories

- `touch` … create file

Example: `touch main.cpp`

- `mkdir` … **m**ake **dir**ectory

Example: `mkdir EC327_Lab1`

- `cp` … **cop**y file

Example: `cp main.cpp main01.cpp`

- `mv` … **mov**e file (also to rename)

Example: `mv main.cpp main01.cpp`

- `rm` … **rem**ove file

Example: `rm main.cpp`

Read Files

- `less <filename>`
- Example: `less stuff.txt`
- Use spacebar to go to next page
- Use “b” to go to **b**ack a page
- Press “q” to **q**uit back to shell
- Press “v” to open file in editor
- To show line numbers, use option –N:

  `less -N <filename>`

Text Editors for Coding

- Why use text editors, not notepad or Word?
- vi is default text editor in UNIX
- Editors allowed for exam
  - vi, vim, emacs, gedit, pluma
- Editors you can use at home
  - VS Code, Sublime, Atom, TextMate...

The vi Editor (*vee-ai*)

Open vi: `vi` [<filename>]

Two modes: EDIT (default) and **INSERT**

`i`	switch to **INSERT** mode
`ESC`	go back to EDIT mode

(some) EDIT mode commands:

`:q`	quit
`:q!`	force quit
`:wq`	write and quit
`dd`	delete line
`5G`	go to line #5 (capital G !)
`u`	undo

http://unixhelp.ed.ac.uk/vi/ref.html

Using the vi Editor

```
vi HelloWorld.cpp

    #include <iostream>

    using namespace std;

    int main()
    {
            cout << "Hello World!" << endl;

            return 1;
    }
```

The vim Editor

- **V**i **IM**proved, everything in Vi and more
- Try `vimtutor` for tutorial
- Resources:
  - openvim.com
  - vim-adventures.com
  - vimcasts.org

The emacs Editor

http://www.gnu.org/software/emacs/

Start emacs: `emacs` [<filename>]

Start emacs to continue working in Shell:

`emacs` [<filename>] **&**

(some) emacs Commands:

^x^s	**Save File**
^x^c	**Quit**

& … start a process (e.g. the emacs Editor) in the background

Using the emacs Editor

```
emacs HelloWorld.cpp
```

```
#include <iostream>

using namespace std;

int main()
{
        cout << "Hello World!" << endl;

        return 1;
}


-uu-:**-F1  main.cpp        All L13    (C++/l Abbrev)---------
```

^x^s … Save

^x^c … Quit

Running the emacs Editor in parallel with the Shell/Terminal

```
emacs HelloWorld.cpp &

ls -l
cd ..
```

```
#include <iostream>

using namespace std;

int main()
{
        cout << "Hello World!" << endl;

        return 1;
}


-uu-:**-F1  main.cpp       All L13    (C++/l Abbrev)---------
```

The Shell/Terminal process and the emacs Process run in parallel.

Humans lose track easily when running multiple processes in parallel!

Security and Access Rights

`ls -l` list files in “long” format

```
localhost:Labs ernstl$ ls -l
total 0
drwxr-xr-x  2 ernstl  staff  68 Sep  6 15:43 Lab1
-rw-r--r--  1 ernstl  staff   0 Sep  6 15:49 inc.h
-rw-r--r--  1 ernstl  staff   0 Sep  6 15:49 main.cpp
-rw-r--r--  1 ernstl  staff   0 Sep  6 15:49 my.h
```

d	Directory
r	**r**ead
w	**w**rite
x	e**x**ecute

d r w x r – x r – x

user group world

Example:

user **ernstl**

group **staff**

Security and Access Rights

`chmod` **ch**ange file **mod**e

`chown` **ch**ange **own**er

We will not discuss those commands in more detail!

However, UNIX provides manuals!

“Man Pages”

- Man is an abbreviation for **Manual**
- `man <command>`
- Opens the manual in the **vi** Editor
- Most programs can also try `<program> --help`

Examples:

```
man ls
man cd
man g++
```

At home, check out tldr.sh and tealdeer

(Un)Zipping (1/2)

- Various compression formats and tools
  - tar
  - gzip
  - **zip** (used in EC327)
- The **zip** command and its arguments:

```
zip <zip-filename> <file>
zip -r <zip-filename> <directory>
```

Important!

While zipping a directory, you CANNOT place the zip-file into the directory you're zipping!

(Un)Zipping (2/2)

Example:

```
mkdir Lab1
cd Lab1
emacs main.cpp
zip -r Lab1.zip Lab1/*
```

Option 1:

Go to the directory above!

```
cd ..
zip -r Lab1.zip Lab1/*
```

Option 2:

Put the zip-file into the above directory!

```
zip -r ../Lab1.zip ../Lab1/*
```

Unzipping:

```
unzip <zip-filename>
```

Zipping
EC327 Midterm and Finals (1/2)

a. Open a terminal window.

b. Create a folder named **<yourBUusername>_<last4digitsofBUID>_<machineNumber>_final**. For example, student *James Bond* with BU username *jbond*, BU ID U*12340007,* and machine number *002* should type:

```
mkdir jbond_0007_002_final
```

*** Make sure you put in your own name and number here, and not those of James Bond's!!!*

c. Change directory to your final directory:

```
cd jbond_0007_002_final
```

d. Create `test.cpp` file in your directory, and write the following code:

```
#include <iostream>
using namespace std;
int main()
{
  cout<< "TEST EC327 FINAL" << endl;
  return 0;
}
```

Zipping
EC327 Midterm and Finals (2/2)

e. Zip your directory and submit via the following steps:

```
cd ..
ls
```

** *You should see your jbond_0007_002_final directory listed now.*

```
zip -r jbond_0007_002_final.zip ./jbond_0007_002_final/*
```

** *Make sure to replace James Bond's folder name with your folder name.*

f. Check that the zip file exists and contains all of the files:

```
ls
```

** *You should see jbond_0007_002_final.zip*

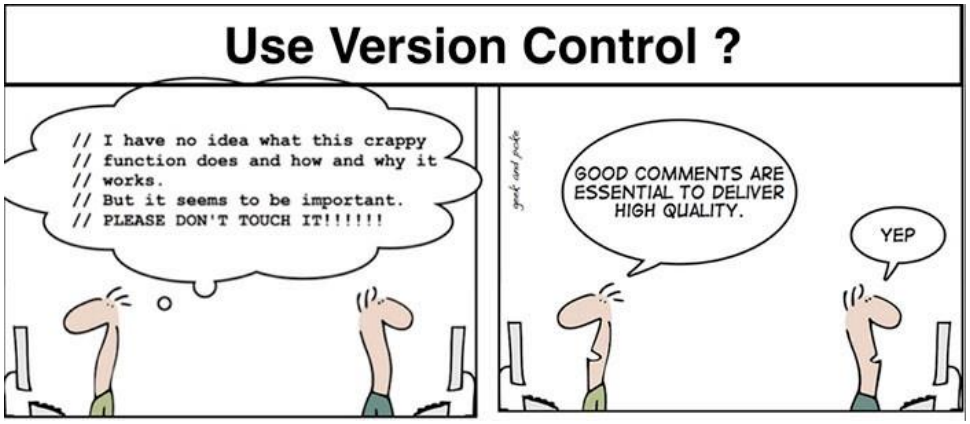
```
less jbond_0007_002_final.zip
```

** *This will list all the files in the zip. Make sure it contains all your files. ('q' to quit "less" program)*

COLLABORATIVE SUMMARY

- What do the following commands stand for, do, and what arguments do they take?

```
ls
cd
mv
man
:q
zip
ls -l
less
```



Version Control

Version Control Quick Intro (1/2)

- In this class you will edit many files
- What's the best way to keep track of **changes**?
  - Creating multiple copies of the same file does not scale
  - Collaborating with others will be difficult
- Like most software developers, we will use a VCS (Version Control System) in this class

Version Control Quick Intro (2/2)

- `git` is a free VCS found on many OS (GNU/Linux, Mac OS, etc..)
- Changes stored in a database-structure called **repository**
- `git` operates on a local repository but this local repository can be synched with a **remote** one
- GitHub is a free repository hosting service
  - We will be using GitHub to submit assignments

GitHub Account

- Make sure that you have a GitHub account
  - https://github.com/join
- You will use this account for future submissions and today's lab

Initializing The Repository (1/2)

- Need a **local** copy of the repository

`git init .` Creates an empty repository

`git clone <address>`

  - Downloads a repository from another computer
  - You will be able to see change history and files
  - Git repositories are stored in a hidden directory `.git/`

Initializing The Repository (2/2)

EC327-Spring-2020 / **github-trial-mmark9**

Unwatch 4 | Star 0 | Fork 0

Code | Issues 0 | Pull requests 0 | Actions | Projects 0 | Wiki | Security | Insights

Quick setup — if you've done this kind of thing before

or HTTPS SSH `https://github.com/EC327-Spring-2020/github-trial-mmark9.git`

Get started by creating a new file or uploading an existing file. We recommend every repository include a README, LICENSE, and .gitignore.

...or create a new repository on the command line

```
echo "# github-trial-mmark9" >> README.md
git init
git add README.md
git commit -m "first commit"
git remote add origin https://github.com/EC327-Spring-2020/github-trial-mmark9.git
git push -u origin master
```

- We can clone remote repository from GitHub via HTTPS
- Clone address will end `.git`
- Try cloning the GitHub trial assignment!

Viewing Changes

- As you edit your files, you should track your changes
- `git status` to view **<u>which</u>** files changed

```
-bash-4.2$ git status
# On branch master
# Changes not staged for commit:
#   (use "git add <file>..." to update what will be committed)
#   (use "git checkout -- <file>..." to discard changes in working directory)
#
#       modified:   README.md
#
no changes added to commit (use "git add" and/or "git commit -a")
```

- `git diff` to show **<u>what</u>** parts of the file changed

Deletions start with -

Additions start with +

```
-bash-4.2$ git diff README.md
diff --git a/README.md b/README.md
index eac6130..3ff4837 100644
--- a/README.md
+++ b/README.md
@@ -10,6 +10,6 @@ After finishing part 1 of lab 3, submit your assignment through github
 4. Commit your changes to local repository: `git commit`.
 5. [optional] Verify your commit is as you expected: `git show`
 6. Push your changes to Github: `git push -u origin master`
-7. ??
-8. Profit
+7. View your commit on Github
+8. Comment on your commit on github
```

Adding Changes

- Now you know your changes, you can add them to the **staging area**
- Think of staging area as a space to prepare your **commit**
- `git add <file1> <file2> …` adds files to the staging area
- `git add <pattern>` will add all files matching a pattern
- `git add *` adds all changed files to staging area

Committing Changes (1/2)

- Changes in your staging area are not applied to your repository
- Must commit changes first to update repository
- `git commit` will launch an editor (vi, vim, nano) so you can write your commit

Committing Changes (2/2)

- Commit message should answer two questions:

```
Add extra credits instructions for students

A student asked if there would be extra credit points for this
assignment. After checking the course grading policy, we have an
option to supplement some assignments with additional points

# Please enter the commit message for your changes. Lines starting
# with '#' will be ignored, and an empty message aborts the commit.
# On branch master
# Changes to be committed:
#   (use "git reset HEAD <file>..." to unstage)
#
#       modified:   README.md
```

What does the commit do?
This is your commit **title**

Why did you make this change?
This is your commit **body**

- `git commit -m "<message>"` will treat your message as the title
  - Useful for small commits

Publishing Changes

- So far our **local** repository is updated but the **remote** isn't
- To update the remote repository we have to **push** our changes

Format for pushing remotely:

```
git push [<repository_id>] [<branch>]
```

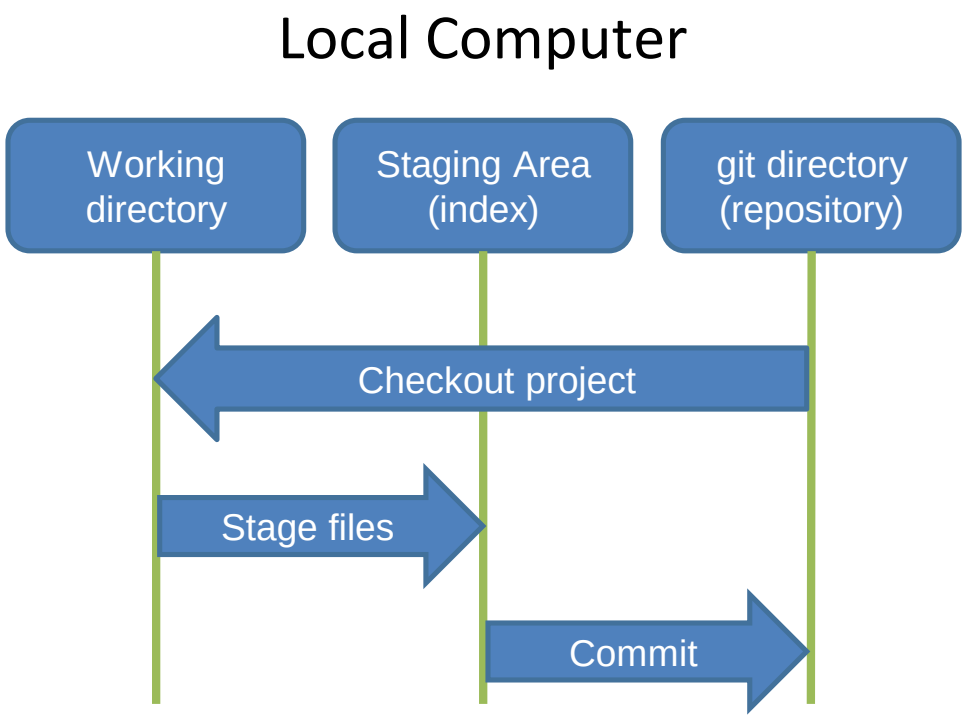
For this lab you can use:

```
git push origin master
```

- Add the `-u` flag the first time you push!

git Usage Summary

- All projects using `git` will follow the same general steps
  1. `git clone` to download a repository from remote server and checkout project
  2. `git add` to prepare changes to be added to local repository - also called ***staging***
  3. `git commit` to commit changes to local repository
  4. `git push` to copy updated local repository to another computer



Resources

UNIX Tutorial for Beginners
http://www.ee.surrey.ac.uk/Teaching/Unix/

Learn UNIX in 10 Minutes
http://freeengineer.org/learnUNIXin10minutes.html

UNIX Tutorial (UC Berkeley)
http://people.ischool.berkeley.edu/~kevin/unix-tutorial/toc.html

Git Tutorial
https://git-scm.com/docs/gittutorial

Just use your favorite Internet Search Engine ☺

Motivation

- **Why using (and learning) UNIX?**
  - Family of OSs
  - Simple and small-scale **kernel**
  - Kernel provides **APIs to interact with hardware**
  - **C** is a language to **program and control systems and hardware**
  - …

Why Shell/Terminal?

- Efficient **Programming Languages are textual**

A computer program is an ordered set of instructions to be executed by a computer.

- "**Shell Scripting**"
  - Every command is a "simple" tool
  - Commands can be grouped together and executed in a desired order (**Workflow**)

Every Computer/Software Engineer should be familiar with UNIX and using the Shell!

QUESTIONS?

Staff

Instructor

Gianluca Stringhini (gian@bu.edu, **PHO 331**)

Office hours: Monday 5-6pm and Wednesday 5-6pm, also by appointment.

Graduate Teaching Fellows

Burak Aksar (baksar@bu.edu); Office Hours: **Wednesday 1-2pm, PHO 307** (and by appt.); F labs

Miguel Mark (mmark9@bu.edu); Office Hours: **Wednesday 2-3pm, PHO 307** (and by appt.); W labs

Undergraduate Teaching Fellows

Benjamin Chan (chanben@bu.edu); Office Hours: **Thursday 1-2pm** (and by appt.); F labs

Neha Rai (nrai@bu.edu); Office Hours: **Friday 4:30-5:30pm** (and by appt.); T labs

Panat Taranat (ptaranat@bu.edu); Office Hours: **Tuesday 5-6pm** (and by appt.); T labs

Angela Xu (angelax@bu.edu); Office Hours: **Tuesday 2-3pm** (and by appt.); W labs